

Pattern Matching with Swaps¹

Amihood Amir²

*Department of Mathematics and Computer Science, Bar-Ilan University, 52900
Ramat-Gan, Israel; and Georgia Tech, Atlanta, Georgia 30332*
E-mail: amir@cs.biu.ac.il

Yonatan Aumann

*Department of Mathematics and Computer Science, Bar-Ilan University, 52900
Ramat-Gan, Israel*
E-mail: aumann@cs.biu.ac.il

Gad M. Landau³

*Department of Computer and Information Science, Polytechnic University, Six
MetroTech Center, Brooklyn, New York 11201-3840; and Department of Computer
Science, Haifa University, Haifa 31905, Israel*
E-mail: landau@poly.edu

and

Moshe Lewenstein and Noa Lewenstein⁴

*Department of Mathematics and Computer Science, Bar-Ilan University, 52900
Ramat-Gan, Israel*
E-mail: moshe@cs.biu.ac.il, noa@cs.biu.ac.il

Received June 24, 1998

¹ A preliminary version of this paper appeared in FOCS 97.

² Partially supported by NSF grant CCR-96-10170, BSF grant 96-00509, and a BIU internal research grant.

³ Partially supported by NSF grants CCR-9305873 and CCR-9610238 and by the Israel Science Foundation founded by the Israeli Academy of Sciences and Humanities.

⁴ Partially supported by the Israel Ministry of Science and Arts Grant 8560.

Let a text string T of n symbols and a pattern string P of m symbols from alphabet Σ be given. A *swapped version* T' of T is a length n string derived from T by a series of *local swaps* (i.e., $t'_\ell \leftarrow t_{\ell+1}$ and $t'_{\ell+1} \leftarrow t_\ell$), where each element can participate in *no more than one swap*. The *pattern matching with swaps* problem is that of finding all locations i for which there exists a swapped version T' of T with an exact matching of P in location i of T' . It has been an open problem whether swapped matching can be done in less than $O(nm)$ time. In this paper we show the first algorithm that solves the pattern matching with swaps problem in time $o(nm)$. We present an algorithm whose time complexity is $O(nm^{1/3} \log m \log \sigma)$ for a general alphabet Σ , where $\sigma = \min(m, |\Sigma|)$. © 2000 Academic Press

Key Words: design and analysis of algorithms; combinatorial algorithms on words; pattern matching; pattern matching with swaps; nonstandard pattern matching.

1. INTRODUCTION

1.1. Background

String matching is one of the most widely studied problems in computer science [9]. Part of its appeal is in its direct applicability to real world problems. The Boyer–Moore [5] algorithm is directly implemented in the *emacs* “s” and UNIX “grep” commands. The longest common subsequence dynamic programming algorithm [6] is implemented in the UNIX “diff” command. The largest overlap heuristic for finding the shortest common superstring [28] has been used in DNA sequencing, and there are many other examples.

Advances in multimedia, digital libraries and computational biology have shown that a much more generalized theoretical basis of pattern matching could be of tremendous benefit [26]. To this end, pattern matching has to adapt itself to increasingly broader definitions of matching. In computational biology one may be interested in finding a close mutation, in communications one may want to adjust for transmission noise, and in texts it may be desirable to allow common typing errors. In multimedia one may want to adjust for lossy compressions, occlusions, scaling, affine transformations, or dimension loss.

All the above applications motivated the *generalized pattern matching* problem. In generalized matching the input is still a text and pattern but the matching relation is defined differently. The output is all locations in the text where the pattern matches under the new definition of match. The different applications define the matching relation. An early generalized matching was defined by Fischer and Paterson [8]. They allow a special “don’t care” character that serves as a wild card. It matches every other alphabet symbol. A text location where the pattern matches under these conditions is an exact match.

Even under the new matching relation there is still a distinction between *exact matching* and *approximate matching*. In the latter case, a metric is defined on the text. A text location is considered a match if the distance between it and the pattern, under the given metric, is within the tolerated bounds.

The fundamental question is what type of approximations is inherently hard computationally and what types are faster to compute. This question motivated much of the pattern matching research in the past 20 years.

The earliest, and by now classic, metrics for approximate matching are the following. Levenshtein [18] identified three types of errors: mismatches, insertions, and deletions. These operations are traditionally used to define the *edit distance* between two strings. Lowrance and Wagner [20, 29] added the *swap* operation to the set of operations defining the distance metric. The swap, where the order of two consecutive symbols is reversed, is one of the most typical typing errors. More complex versions of the swap occur in nature. The phenomenon of swaps occurs in gene mutations and duplications (e.g., the region of human chromosome 5 that is implicated in the disease called spinal muscular Atrophy, a common recessive form of muscular dystrophy [19]). While the biological swaps occur at a gene level and have several additional constraints and characteristics, which make the problem much more difficult, they do serve as a convincing pointer to the theoretical study of swaps as a natural edit operation for the approximation metric.

Using the edit distance or extended edit distance as a metric, one may then define a match to be a text location where the pattern matches with no more than a given number of allowed operations.

The lower bound story is rather short. Muthukrishnan and Ramesh [24] prove that practically all general matching relations, where the generalization is in the definition of single symbol matches, are equivalent to the Boolean convolutions; i.e., it is unlikely that they could be solved in time faster than $O(n \log m)$, where n is the text length and m is the pattern length. Furthermore, Muthukrishnan and Palem [23] proved that in the *Boolean convolutions model* there are nontrivial lower bounds on the number of necessary convolutions for several nonstandard string matching problems.

Most research success in generalized and approximate pattern matching lies with faster upper bounds. Let n be the text length and m the pattern length. Lowrance and Wagner proposed an $O(nm)$ dynamic programming algorithm for the extended edit distance problem. In [11, 16, 17] $O(kn)$ algorithms are given for the edit distance with only k allowed edit operations. Sahinalp and Vishkin [27] presented an $O(nk^{8+1/\log 3} (\frac{\log^* n}{n})^{1/\log 3})$ algorithm for this problem. A faster algorithm was presented by Cole and Hariharan [7] with $O(nk^4/m + n)$ running time.

Since the upper bound for the edit distance seems very tough to break, attempts were made to consider the edit operations separately. If only mismatches are counted for the distance metric, we get the *hamming distance*, which defines the *string matching with mismatches* problem. A great amount of work was done on finding efficient algorithms for string matching with mismatches [1, 4, 10, 13–15].

1.2. Isolating the Swap Operation

In this paper we attempt to isolate the *swap* operation from the rest of the extended edit operations, in the manner in which the *mismatch* operation had been isolated in the past, to enable careful analysis of the inherent difficulty of the swap operation alone. We present the first algorithm whose complexity beats the naive $O(nm)$ bound.

The existing lower bounds in nonstandard stringology do not apply in the case of the *string matching with swaps* problem, because of its contextual nature. The only exception is a new result reducing Boolean convolutions to string matching with swaps [22]. However, there were no known upper bounds better than the naive $O(nm)$ algorithm. Indeed, this problem was formulated and described as one of the open problems in nonstandard string matching [21].

We tackle the problem by first finding a solution to the generalized matching problem with swaps (see [21]). In this problem one seeks all text locations where the pattern matches allow swaps in the text, regardless of the number of swaps. We present an algorithm whose time complexity is $O(nm^{1/3} \log m \log \sigma)$ for a general alphabet Σ , where $\sigma = \min(m, |\Sigma|)$.

Our method is as follows. We first reduce the general alphabet swapped matching problem to a two letter alphabet problem with a complexity penalty of only an $O(\log \sigma)$ multiplicative factor, where $\sigma = \min(m, |\Sigma|)$. We then concentrate our efforts on the swapped matching problem over alphabet $\Sigma = \{a, b\}$. We prove a structural concatenation lemma that defines the conditions under which it is permissible to group together swapped matches of consecutive pattern substrings to get a swapped match of the entire pattern. We show that the less-than matching problem [2] can be used to efficiently handle the concatenation conditions. Finally, we solve our special case of less-than matching with “don’t cares” in time $O(nm^{1/3} \log m)$.

Now that we have an efficient solution to the swapped matching problem as a generalized matching problem, we consider the number of swaps as a distance metric. We show how our algorithm can be used to provide a $O(n\sqrt{m} \text{ polylog}(m))$ algorithm for the approximate string matching problem where the distance metric is the number of swaps.

1.3. Paper Organization

This paper is organized in the following way. In Section 2 we give basic definitions and prove some crucial traits of the problem. In Section 3 we prove that reduction to a two-symbol alphabet merely causes a multiplicative $\log \sigma$ degradation in time complexity. In Section 4 we concentrate on solving the swapped matching problem over alphabet $\Sigma = \{a, b\}$ and achieve a $O(nm^{1/3} \log m)$ algorithm. Coupled with the result of Section 3 this gives a solution to the swapped matching problem over general alphabets in time $O(nm^{1/3} \log m \log \sigma)$, where $\sigma = \min(m, |\Sigma|)$.

2. PROBLEM DEFINITION AND PRELIMINARIES

DEFINITION. Let $T = t_1, \dots, t_n$ be a *text* string over alphabet Σ . A *swap permutation* for T is a permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ such that

1. if $\pi(i) = j$ then $\pi(j) = i$ (characters are swapped).
2. for all i , $\pi(i) \in \{i-1, i, i+1\}$ (only adjacent characters are swapped).
3. if $\pi(i) \neq i$ then $t_{\pi(i)} \neq t_i$ (identical characters are not swapped).

For a given swap permutation π and text T we denote $\pi(T) = t_{\pi(1)}, t_{\pi(2)}, \dots, t_{\pi(n)}$. We call $\pi(T)$ a *swapped version* of T . For pattern $P = p_1, \dots, p_m$ and text T , we say that P has a *swapped match (SM)* at location i if there exists a swapped version T' of T such that P has an exact match with T' starting at location i , i.e., $p_j = t'_{i+j-1}$ for $j = 1, \dots, m$.

The *pattern matching with swaps problem* is the following:

Input: Text string $T = t_1, \dots, t_n$ and pattern string $P = p_1, \dots, p_m$ over alphabet Σ .

Output: All locations i where P has a swapped match in T .

For ease of the exposition we consider the following restriction on the matching.

Restriction. We say that pattern P has a *swapped match with internal starting symbol (SM_{sin})* at location i if P exactly matches a swapped version $\pi(T)$ starting at location i and $\pi(i) \geq i$ (i.e., symbol i was not swapped to outside the pattern). Note that any swapped match either has a swapped

match with internal starting symbol or $p_2 \dots p_m$ does. Therefore, it follows that:

LEMMA 1. *If the swapped pattern matching with internal starting symbol problem can be solved in time $O(f(n, m))$ then the pattern matching with swaps problem can also be solved in time $O(f(n, m))$.*

Further restriction. We further add the restriction that the *ending symbol* is not allowed to swap externally; i.e., $\pi(i + m - 1) \leq i + m - 1$. We call a match for which both the starting symbol and the ending symbol do not swap externally a *swapped match with internal endpoints* (SM_{in}).

In a similar manner to Lemma 1, one can show that any solution of time complexity $O(f(n, m))$ for the swapped pattern matching with internal endpoints problem can be used to derive a solution for the general swapped matching problem in time $O(f(n, m))$. From now on, in all our algorithms we use any of the internal endpoints versions of the problem as convenience dictates.

Uniqueness of the swap permutation. For a string $S = s_1 \dots s_t$ we denote $S[i, j] = s_i \dots s_j$. The following lemma proves the uniqueness of the swap permutation in the case of internal starting point swapped match. The uniqueness is needed in Section 3 where we show the reduction of general alphabets to the binary alphabet case.

LEMMA 2. *Consider an alphabet Σ , text T , and pattern P , $|P| = m$. If there exists a swapped match with internal starting symbol of P in T at location i then the swap permutation providing this match is unique on the set $\{i, \dots, i + m\}$; i.e., there exists a unique swap permutation $\pi : \{i, \dots, i + m\} \rightarrow \{i, \dots, i + m\}$ such that $t_{\pi(\ell)} = p_\ell$ for $\ell = i, \dots, i + m - 1$*

Proof. Since the match is with an internal starting point we may assume w.l.o.g. that $i = 1$. We prove by contradiction.

Say m is the smallest length of a pattern P for which this does not hold. It is easy to verify that $m > 1$. Let π and τ be two different swap permutations producing this match. Consider the restriction, π' and τ' , of π and τ (respectively) to $\{1, \dots, m\}$. Similarly, let π'' and τ'' be the restriction of π and τ (respectively) to $\{1, \dots, m - 1\}$. Note that π' and τ' both produce a match for $P[1, m - 1]$, and π'' and τ'' both produce a match for $P[1, m - 2]$. Also note that, in general, the restrictions $\pi', \tau', \pi'', \tau''$ are not necessarily permutations. There are three possible cases:

1. $\pi(m + 1) = \tau(m + 1) = m + 1$: then both π' and τ' are permutations. Hence, by minimality of m , $\pi' = \tau'$. Thus, $\pi = \tau$, a contradiction.

2. $\pi(m + 1) = \tau(m + 1) = m$: then $\pi(m) = \tau(m) = m + 1$ and both π'' and τ'' are permutations. Thus, by minimality of m , $\pi'' = \tau''$ and $\pi = \tau$, a contradiction.

3. $\pi(m+1) = m+1$, $\tau(m+1) = m$: then π' is a permutation. Moreover, since we do not swap identical symbols and $\tau(m) \neq m$, it must be that $p_m \neq t_m$. Hence $\pi'(m) = m-1$. Define the following swap permutation $\hat{\tau}$ on $\{1, \dots, m\}$: $\hat{\tau}(i) = \tau(i)$, for $i < m$ and $\hat{\tau}(m) = m$ (this is well defined since $\tau(m) = m+1$). Note that $\hat{\tau}(T[1, m-1]) = \tau(T[1, m-1]) = P[1, m-1]$. Thus, both π' and $\hat{\tau}$ are swap permutations for $P[1, m-1]$. However $\pi'(m) \neq \hat{\tau}(m)$. Thus, $\pi' \neq \hat{\tau}$, in contradiction to the minimality of m . ■

When Lemma 2 is applied on P and T and then on P^R and T^R the following is yielded.

COROLLARY 1. *For any alphabet Σ , text T , and pattern P , if there exists a swapped match with internal endpoints of P in T at location i then the swap permutation providing this match is unique on the set $\{i, \dots, i+m-1\}$.*

3. REDUCING GENERAL ALPHABETS TO A TWO LETTER ALPHABET

In this section we show how to reduce the swapped matching problem over unbounded alphabets to the problem over a two letter alphabet. The reduction entails an $O(\log |\Sigma|)$ multiplicative overhead, where $|\Sigma|$ is the size of the alphabet. Here, we are assuming that $|\Sigma| \leq m+1$. Otherwise, change all text symbols that are not in the pattern to a single symbol that does not appear in the pattern.

DEFINITION. Let Σ be an alphabet, and let $F = \{\chi_1, \dots, \chi_k\}$ be a family of characteristic functions, $\chi_j : \Sigma \rightarrow \{0, 1\}$. We say that F is a *separating family* if for each ordered triplet of characters $(a, b, c) \in \Sigma^3$ there exists a χ_j such that $\chi_j(a) \neq \chi_j(b) = \chi_j(c)$.

A natural separating family is the set of *projections* $\{\chi_\sigma\}_{\sigma \in \Sigma}$, where

$$\chi_\sigma(x) = \begin{cases} 1 & \text{if } x = \sigma \\ 0 & \text{if } x \neq \sigma. \end{cases}$$

We extend the definition of the functions χ_j to a string in the usual manner; i.e., for $S = s_1 s_2 \dots s_n$, $\chi_j(S) = \chi_j(s_1) \chi_j(s_2) \dots \chi_j(s_n)$.

THEOREM 1. *Let P be a pattern and T a text, both over an arbitrary alphabet Σ , and let $F = \{\chi_1, \dots, \chi_k\}$ be a separating family for Σ . There is a SM_{sin} for P in T at location i iff for all j there is a SM_{sin} of $\chi_j(P)$ in $\chi_j(T)$ at location i .*

Proof. W.l.o.g. $i = 1$.

Only if: The swap permutation providing the match for P in T yields a match for $\chi_j(P)$ in $\chi_j(T)$ for all j (technically, we eliminate from the permutation the swaps which swap identical symbols in $\chi_j(T)$).

If: We show that if there is no SM_{sin} for P in T (at location 1) then there exists a j for which there is no SM_{sin} . The proof is by induction on ℓ , the length of the largest prefix $P[1, \ell]$ of P , for which there exists a SM_{sin} of $P[1, \ell]$ in T (at location 1). For this ℓ we show that there exists a j for which there is no SM_{sin} of $\chi_j(P[1, \ell + 1])$ in $\chi_j(T)$ at location 1. Thus, there is also no SM_{sin} of $\chi_j(P)$ in $\chi_j(T)$ at location 1.

Base case $\ell = 0$: This happens in case $P[1, 1]$ does not have a SM_{sin} at location 1. Thus, $p_1 = a \neq b = t_1$ and $t_2 = c \neq a$. By definition of a separating family there exists a j such that $\chi_j(a) \neq \chi_j(b) = \chi_j(c)$. For this j there is no SM_{sin} of $\chi_j(P[1, 1])$ in $\chi_j(T)$ at location 1.

Inductive step: Assume that $P[1, \ell]$ is the longest prefix of P that has a SM_{sin} at location 1. Let π be the permutation that provides the match for $P[1, \ell]$. The following cases need to be considered:

1. π does not swap t_ℓ to the right (externally). Let $p_{\ell+1} = a$, $t_{\ell+1} = b$, and $t_{\ell+2} = c$.

(a) $a = b$ or $a = c$: in this case $P[1, \ell + 1]$ has a SM_{sin} , violating the maximality of ℓ .

(b) $a \neq b$ and $a \neq c$: by assumption there exists a j such that $\chi_j(a) \neq \chi_j(b) = \chi_j(c)$. For this j there is no SM_{in} for $\chi_j(P[1, \ell + 1])$ in $\chi_j(T)$ at location 1. This is because if there were a match, then, by the uniqueness of the swap permutations (Lemma 2), the swap permutation, τ , providing such a match must agree with π of $T[1, \ell]$. Thus, $\chi_j(p_{\ell+1})$ must match $\chi_j(t_{\ell+1})$ or $\chi_j(t_{\ell+2})$, which it does not.

2. π swaps t_ℓ to the right. Let $p_\ell = t_{\ell+1} = a$, $t_\ell = b$, and $p_{\ell+1} = c$.

(a) $b = c$: then $P[1, \ell + 1]$ has a SM_{sin} , violating the maximality of ℓ .

(b) $b \neq c$: let j be such that $\chi_j(b) \neq \chi_j(a) = \chi_j(c)$. For this j there is no SM_{in} for $\chi_j(P)$ in $\chi_j(T)$ at location 1 (the reasoning is similar to that of 1(b)). ■

We now show how to construct a small separating family for an alphabet.

DEFINITION. An (n, k) -universal set is a set S of n -digit binary vectors such that for any set of positions $L \subset \{1, \dots, n\}$, $|L| = k$, the projection of S on L gives all possible 2^k combinations of 0 – 1s.

In [25] it is shown how to construct an (n, k) -universal set of cardinality $2^{O(k)} \log n$ with the same time bounds. Note that a $(|\Sigma|, 3)$ -universal set is a separating family for Σ . This yields the following corollary.

COROLLARY 2. *If the swapped pattern matching (with internal endpoints) problem can be solved over alphabet $\{a, b\}$ in time $O(f(n, m))$ then the swapped pattern matching (with internal endpoints) problem can be solved in time $O(\log |\Sigma| f(n, m))$ over a general alphabet Σ .*

Proof. This follows from Theorem 1 and the existence of an $O(\log |\Sigma|)$ size separating family. ■

4. SWAPPED MATCHING OVER A TWO LETTER ALPHABET

Following Corollary 2, we now only consider a two letter alphabet $\Sigma = \{a, b\}$.

DEFINITION. Let $S = s_1 \dots s_m$ be a string. We say that S is *alternating* if $\forall i > 1, s_{i-1} \neq s_i$. For a given pattern P we say that $P[i, j]$ is a *maximal alternating substring* of P if $P[i, j]$ is alternating and no other alternating substring of P strictly includes locations i through j in P .

Since P is a pattern over $\{a, b\}$, it is obvious that there is a unique partition $P = P^1 P^2 \dots P^t$ such that each P^j is a maximal alternating substring of P . This partition can easily be constructed in $O(n)$ time.

To find the match of P in T our algorithm checks for a match of each maximal alternating substring separately. The following lemma shows that this is sufficient.

LEMMA 3 (Concatenation Property). *Let P be a pattern and $P = P^1 \dots P^t$ be the partition of P into maximal alternating substrings. For text T , P has a swapped match with internal endpoints at location i iff for each $j = 1, \dots, t$, P^j has a swapped match with internal endpoints in T at the corresponding locations (i.e., at location $i + \sum_{\ell=1}^{j-1} |P^\ell|$).*

Proof. Denote $m_j = |P^j|$ and $i_j = i + \sum_{\ell=1}^{j-1} m_\ell$.

If: Let π_j be the swap permutation providing the SM_{in} of P^j in T at location i_j . We construct a joint swap permutation π for P by pasting the segments of the π_j 's on the P^j 's. Formally, $\pi(k) = \pi_j(k)$ for $i_j \leq k < i_{j+1}$ and $\pi(k) = k$ for all other k . Since the π_j are internal endpoints swap permutations, π is also an internal endpoints swap permutation for P .

Only if: Let π be the swap permutation providing the SM_{in} of P in T at location i . We show that for each j , π provides a SM_{in} for P^j at location i_j . In contradiction assume that π does not provide a SM_{in} for P^j . Clearly π provides a regular swapped matching for P^j in T at location i_j . Thus, it must be that π swaps either i_j with $i_j - 1$ or $i_j + m_j - 1$ with $i_j + m_j$.

The first case would only happen if $p_{i_j} \neq p_{i_{j-1}}$ in which case P^j could be extended one symbol to the left for a longer alternating substring in contradiction to the maximality of P^j . The second case is similar. ■

It thus remains to show how to check that all maximal substrings have an SM_{in} . In the next section we present the notion of *streams* and their construction. The streams are the fundamental building block in the algorithm for finding the matches of alternating substrings.

4.1. Streams

Given a text T and a pattern P , we are interested in finding all SM_{in} of maximal alternating substrings of P in T . As a first step we ignore P and for each location i in T find the longest alternating string that has a SM_{sin} at this location. To this end we define the notion of *streams*.

DEFINITION. For text T a *stream* S starting at location i is an alternating string S with a SM_{sin} at location i . A stream S starting at location i is *maximal* if there is no other stream S' at location $i' \leq i$ such that S is a proper substring of S' , and S and S' agree on the overlapping locations, i.e., $s_j = s'_{i-i'+j}$, $j = 1, \dots, |S|$. The following is an example of a maximal stream starting at location 1. The arrows show the swap permutation providing the match.

EXAMPLE.

$$\begin{array}{cccccccccccccccc} S_1 = & a & b & a & b & a & b & a & & & & & & & & & \\ T = & a & \overrightarrow{a} & \overleftarrow{b} & \overrightarrow{a} & \overleftarrow{b} & \overrightarrow{a} & \overleftarrow{b} & a & a & b & a & b \end{array}$$

We distinguish between two types of streams. In the *odd-a type* stream, the a 's of the stream match the odd locations of the text, and in the *even-a type* streams, the a 's match the even text locations. The above example is of an odd- a type stream. Generally, there may be more than one maximal stream of the same type at any text location. This can be seen in the following example.

EXAMPLE.

$$\begin{array}{cccccccccccccccccccc} S_1 = & a & b & a & b & a & & & & & & & & & & & a & b & = & S_3 \\ T = & a & \overrightarrow{a} & \overleftarrow{b} & \overrightarrow{a} & \overleftarrow{b} & \overleftarrow{a} & a & \overrightarrow{a} & \overrightarrow{b} & \overleftarrow{a} & b & b \\ S_2 = & & & \overrightarrow{a} & \overleftarrow{b} & \overrightarrow{a} & \overleftarrow{b} & a & \overrightarrow{b} & \overleftarrow{a} & \overrightarrow{b} & \overleftarrow{a} & b \end{array}$$

In this example the text appears in the middle line. S_1, S_2 , and S_3 are maximal streams of type odd- a . The arrows provide the corresponding swap permutation (no arrow indicates an identity). Note that in locations 3, 4, and 5 of the text there is an overlap of the two different streams S_1 and S_2 and in locations 9 and 10 there is an overlap between S_2 and S_3 .

Fortunately, it turns out that there can be no more than two different overlapping maximal streams of the same type at any text location.

LEMMA 4. *For any location i in the text and a given stream type, there are at most two distinct maximal streams of that type that cover location i .*

Proof. In contradiction, assume that S_1 , S_2 , and S_3 are distinct maximal streams of the same type all covering the same location. Let i_1 , i_2 , and i_3 be the starting points and j_1 , j_2 , and j_3 the ending points of S_1 , S_2 , and S_3 , respectively. Note that since all three streams are of the same type they correspond on overlapping locations. Thus, since they are all maximal, we may assume w.l.o.g. that $i_1 < i_2 < i_3$ and $j_1 < j_2 < j_3$ (otherwise one string would subsume the other). Let π_1 , π_2 , π_3 be the swap permutations for S_1 , S_2 , S_3 , respectively. Consider the following two cases:

1. Both π_1 and π_2 swap i_3 with $i_3 - 1$. Then we can combine π_1 up to i_3 with π_2 from $i_3 + 1$ and so on and obtain a swap permutation for a stream from i_1 to j_2 , violating the maximality of S_1 , S_2 .
2. Either π_1 or π_2 do not swap i_3 to the left. W.l.o.g. assume it is π_2 . Note that since the match of S_3 is with an internal starting symbol, by definition π_3 does not swap i_3 to the left. Thus, we may combine π_2 up to $i_3 - 1$ with π_3 from i_3 and so on and obtain a swap permutation for a stream from i_2 to j_3 , violating the maximality of S_2 , S_3 . ■

Thus, we have the following algorithm for finding all streams.

ALGORITHM FOR FINDING ALL STREAMS. Run through the text from left to right. Keep a list of ‘‘active’’ streams. At each location location i .

1. Extend each of the active streams if possible.
2. Start all new possible streams.

End Algorithm

Note that Lemma 2 guarantees that at each location there is only one possible extension of a given stream. Thus, by Lemma 4 the time complexity of the algorithm is $O(n)$.

Once we have the maximal streams of T , we can easily determine for each location i in T the longest stream of a given type that has a SM_{sin} starting at this location. Thus, for each i we define $odd-a(T, i) = (\text{maximal length of odd-a stream starting at location } i)$ and $even-a(T, i) = (\text{maximal length of even-a stream starting at location } i)$.

For example, in the above example $odd-a(T, 2) = 4$, $odd-a(T, 3) = 10$, and $odd-a(T, 9) = 2$. Given the set of all maximal streams we can determine

$odd-a(T, i)$ and $even-a(T, i)$ for all i in $O(n)$ steps. Next we produce two new strings

$$\begin{aligned} OddEven(T) &= odd - a(T, 1), even - a(T, 2), \dots, \\ &\quad odd - a(T, 2i - 1), even - a(T, 2i), \dots \\ EvenOdd(T) &= even - a(T, 1), odd - a(T, 2), \dots, \\ &\quad even - a(T, 2i - 1), odd - a(T, 2i), \dots \end{aligned}$$

$OddEvenT$ and $EvenOdd(T)$ are the basis for the algorithm for finding the partitioned $SM_{\sin}$ match, as described in the next section.

Note that $odd-a(T, i)$ is only an *upper bound* on the length of an odd- a stream that matches at location i . In fact, for any $k \leq odd-a(T, i)$ there is a $SM_{\sin}$ for an odd- a stream of length k at location i . Similarly for even- a type streams.

4.2. Determining $SM_{\sin}$ for All Maximal Alternating Substrings of a Pattern

Consider a text T and pattern P . Let $P = P^1 \dots P^t$ be the partition of P into maximal alternating substrings. We now show how to find all locations i for which, if P is placed over T at location i , each P^j has a $SM_{\sin}$ starting at its corresponding location (formally, all locations i , for which for each j , P^j has a $SM_{\sin}$ starting at location $i + \sum_{\ell=1}^{j-1} |P^\ell|$). We call such a match a *partitioned $SM_{\sin}$ match*. Note that this type of match is almost sufficient for applying the concatenation of Lemma 3. There we need an $SM_{\sin}$ for all P^j 's rather than a $SM_{\sin}$. In the next section we show how to make this final step.

Let $OddEven(T)$ and $EvenOdd(T)$ be as defined in the previous section. Set $m_j = |P^j|$. For the pattern P and substring P^j set $\alpha_j = m_j$ if P^j starts with an a and $\alpha_j = \phi$ otherwise. Set $\beta_j = m_j$ if P^j starts with a b and $\beta_j = \phi$ otherwise. For the pattern P define $P^{start-a} = \alpha_1 \phi^{m_1-1} \alpha_2 \phi^{m_2-1} \dots \alpha_t \phi^{m_t-1}$, $P^{start-b} = \beta_1 \phi^{m_1-1} \beta_2 \phi^{m_2-1} \dots \beta_t \phi^{m_t-1}$, where ϕ^ℓ is the string of ℓ ϕ 's, and ϕ is a "don't care" symbol.

LEMMA 5. *Let P be a pattern and T a text. Consider location i . There is a partitioned $SM_{\sin}$ match for P in T at location i , iff for all $k = 1, \dots, m$:*

1. $(P^{start-a})_k = \phi$ or $(P^{start-a})_k \leq (OddEven(T))_{i+k-1}$, and
2. $(P^{start-b})_k = \phi$ or $(P^{start-b})_k \leq (EvenOdd(T))_{i+k-1}$.

Proof. The pattern locations which are ϕ always have a match. Consider the partition P^j , and suppose that P^j starts with an a . Let $i_j = i + \sum_{\ell=1}^{j-1} m_\ell$. If i_j is odd then a $SM_{\sin}$ match for P^j at location i_j corresponds to odd- a stream starting at location i_j . Thus, there is a $SM_{\sin}$ for P^j at the location i_j

iff $(P^{start-a})_{i_j} = m_j \leq odd-a(T, i_j) = (OddEven(T))_{i_j}$. Similarly, if i_j is even then there is a match iff $(P^{start-a})_{i_j} \leq even-a(T, i_j) = (OddEven(T))_{i_j}$. If P^j starts with a b then the same hold with regards to $EvenOdd(T)$. ■

Thus, we have reduced the problem to that of two *less-than matching* problems. The *less-than matching* problem is defined as follows [2]:

Input: Text $T = t_1, \dots, t_n$, pattern $P = p_1, \dots, p_m$, $t_i, p_i \in N \cup \{\phi\}$.

Output: All locations i for which $p_k = \phi$ or $p_k \leq t_{i+k-1}$, for all $k = 1, \dots, m$.

We have obtained the following algorithm for finding all locations for which there is a partitioned SM_{sin} match of P in T .

BEGIN ALGORITHM 1. Compute all the less-than matches of:

- (a) $P^{start-a}$ and $OddEven(T)$
- (b) $P^{start-b}$ and $EvenOdd(T)$.

2. For each $i = 1, \dots, n - m + 1$ do

If at location i both $P^{start-a}$ is less-than $OddEven(T)$
and $P^{start-b}$ less-than $EvenOdd(T)$, then output i .

End algorithm

In Section 5 we show how to take advantage of our special case, which has a large number of ϕ 's in the pattern, in order to compute the necessary less-than matchings in $O(nm^{1/3} \log m)$ steps.

4.3. From Partitioned SM_{sin} Match to a Full SM_{in} Match

In the previous section we showed how to find all locations for which there is a partitioned SM_{sin} of P in T . This is *almost* what we need for applying Lemma 3, but not quite. Lemma 3 requires that each partition have a SM_{in} , rather than a SM_{sin} . We now show how to isolate the locations for which the partitioned SM_{sin} match is actually a SM_{in} match for each partition. The following immediate lemma provides the basic tool:

LEMMA 6. Let T be a text and S a string, with $|S| = m$, and suppose that S ends with an a . If there is a SM_{sin} for S in T at location i then

$$(\text{number of } a\text{'s in } S) \geq (\text{number of } a\text{'s in } T[i, i + m - 1])$$

and equality holds iff the match is a SM_{in} . The same hold also if S ends with b , with the weak inequality reversed.

Thus, in order to check that the partitioned SM_{sin} match is actually a SM_{in} match we only have to check that the number of a 's (b 's) in the corresponding substrings is identical. Note that since the inequality is always in one direction, an excess in one substring cannot be canceled by another. Thus, it is sufficient to count the total number of a 's (b 's) in the corresponding sections. This we can easily do with two convolutions, as follows.

Given $P = P^1 \dots P^l$, we define $a\text{-end}(P)$ to be the string obtained from P by replacing all symbols of a -ending P^j 's with 1's and of b -ending symbols with 0's. Let $b\text{-end}(P)$ be the bitwise complement of $a\text{-end}(P)$. Now, we perform the convolution of T (with a 's replaced by 1's and b 's by 0's) with $a\text{-end}(P)$ and $b\text{-end}(P)$. Let $A(a)$ be the total number of a 's in the a -ending P^j 's, and let $B(a)$ be the total number of A 's in the b -ending P^j 's. At any location i , the convolution of T and $a\text{-end}(P)$ is exactly $A(a)$ iff for each a -ending P^j the number of a 's in P^j is equal to that in the corresponding locations of T . Similarly, the convolution of T and $b\text{-end}(P)$ is exactly $B(a)$ iff for each b -ending P^j the number of a 's in P^j is equal to that in the corresponding locations of T . Combining Lemma 6 with Lemma 3, we obtain:

COROLLARY 3. *For text T pattern P and location i , P has a SM_{in} in T at location i iff: (i) P has a partitioned SM_{sin} in T at location i , (ii) the convolution of $a\text{-end}(P)$ and T at location i is $A(a)$, and (iii) the convolution of $b\text{-end}(P)$ and T at location i is $B(a)$.*

4.4. Putting It All Together

We summarize the steps of the algorithm:

Input: Text T , pattern P .

Output: All locations i for which P has a SM_{in} in T .

ALGORITHM 1. Partition P into maximal alternating substrings. Time: $O(n)$.

2. Find streams and create $EvenOdd(T)$ and $OddEven(T)$ (Section 4.1). Time: $O(n)$.

3. Using $EvenOdd(T)$ and $OddEven(T)$ find all locations i for which P has a partitioned SM_{sin} match in T (Section 4.2). Time: $O(nm^{1/3} \log m)$.

4. Using convolutions isolate from locations found in step 3, the locations for which there is a SM_{in} (Section 4.3). Time: $O(n \log m)$.

End algorithm.

Thus we have obtained:

THEOREM 2. *The swapped match problem can be solved in $O(nm^{1/3} \times \log m)$ steps over a constant size alphabet and $O(nm^{1/3} \log m \log \sigma)$ steps over a general alphabet Σ , where $\sigma = \min(m, |\Sigma|)$.*

5. LESS-THAN MATCHING

The *less-than matching with don't cares problem* is defined as follows:

Input: Text string $T = t_0, \dots, t_{n-1}$ and pattern string $P = p_0, \dots, p_{m-1}$ where $t_i, p_i \in N \cup \{\phi\}$.

Output: All locations i in T where, for all $k = 1, \dots, m$, $t_{i+k-1} = \phi$, $p_k = \phi$, or $t_{i+k-1} \geq p_k$. In words, every matched element of the pattern is not greater than the corresponding text element, unless either of them is a *don't care* element.

Assume that there are g elements in the pattern that are *not* ϕ (i.e., $m - g$ elements are ϕ). A minor adjustment to the algorithm of [2] allows us to solve the less-than matching with don't cares problem in time $O(\sqrt{g} n \log m)$.

Notation. For $\sigma \in N, x \in N \cup \{\phi\}$ let

$$\chi_\sigma(x) = \begin{cases} 1 & \text{if } x = \sigma \\ 0 & \text{if } x \neq \sigma \end{cases}$$

$$\chi_{<\sigma}(x) = \begin{cases} 1 & \text{if } x < \sigma \\ 0 & \text{if } x \geq \sigma \text{ or } x = \phi. \end{cases}$$

If $X = x_1, \dots, x_n$ then $\chi_\sigma(X) = \chi_\sigma(x_1), \dots, \chi_\sigma(x_n)$. Similarly define $\chi_{<\sigma}(X)$.

We would like to know where each non- ϕ element of the pattern is lined up with something less than it. We can achieve this by computing, for each σ in P , $\chi_{<\sigma}(T) \otimes \chi_\sigma(P^R)$ (where $\otimes$ is polynomial multiplication) and considering all nonzero locations.

Let $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_g\}$ be the set of all different numbers appearing in P . Let $M_i = \chi_{<\sigma_i}(T) \otimes \chi_{\sigma_i}(P^R)$ (where $\otimes$ is polynomial multiplication). Then M_i is nonzero at position t iff there is a σ_i in the pattern matched with something smaller than σ_i when the pattern is lined up at t . These cases are exactly when we get a mismatch. If we let M be the sum of all the M_i 's we get a nonzero if there was a mismatch caused by any $\sigma \in \Sigma$. By using FFT we can calculate each of the polynomial multiplications in

time $O(n \log m)$ (assuming a computer word of $O(\log m)$ bits), for a total of $O(g n \log m)$.

Following [1, 14], Amir and Farach [2] use the multiplication technique on a limited alphabet of size $\sqrt{m}$ and then fine tune by another method. We point out that in reality the limited alphabet is of size $\sqrt{g}$ and the fine tuning algorithm works on blocks of size $O(\sqrt{g})$.

ALGORITHM Input $T = t_0, \dots, t_{n-1}; P = p_0, \dots, p_{m-1}$. Assume that the text alphabet is the same as the pattern alphabet (this can be easily achieved by replacing every text number by the largest pattern number that does not exceed it).

1. For every $a \in \Sigma$ that appears more than $\sqrt{g}$ times in P (a *frequent element*), use FFT to disqualify all text locations where there is a text element that is smaller than its corresponding pattern element.

2. Let $p_{j_0}, \dots, p_{j_{g_1}}$ be the nonfrequent, non- ϕ elements of P . Consider $L = \langle p_{j_0}, j_0 \rangle, \langle p_{j_1}, j_1 \rangle, \dots, \langle p_{j_{g_1}}, j_{g_1} \rangle$. [Every non- ϕ pattern element is considered a pair $\langle s, d \rangle$ where s is a number and d is the location of the number in the array P .]

3. Sort L lexicographically. [There are no more than g elements in L .] Call the sorted array L' .

4. Divide L' into $g_2 \leq \sqrt{g}$ blocks, each containing no more than $2\sqrt{g}$ elements, in a manner such that no number appears in more than one block. [We are assured that such a division is possible because all remaining numbers are nonfrequent.]

5. For each block B_i , $i = 0, \dots, g_2 \leq \sqrt{g}$, let b_i be the smallest (leftmost) element in the block; call b_i the *representative* of block B_i .

6. Let T' and P' be T and P such that every t_i and p_i is replaced by the representative of the block it is in. [Implemented by a sequential scan of L' .]

7. Find all less-than matches of P' in T' .
[P' and T' can be considered "flattened out" versions of P and T . When we seek all less-than matches of P' in T' we only detect the large mismatches, i.e., those between elements that are so different that they are in different blocks. However, mismatches between elements of the same block are undetected. At this stage we must fine tune our approximate solution. We scan T and for every element t_i of T we only compare it to the $O(\sqrt{g})$ elements of P that are in t_i 's block.]

8. For $i = 0$ to $n - 1$ if $t_i \neq \phi$ do
 - Let B_{t_i} be the block of L' that t_i is in,
 - Let $P^{B_{t_i}} \leftarrow \{ \langle s, d \rangle \mid \langle s, d \rangle \in B_{t_i} \}$
 - For every element $\langle s, d \rangle$ in $P^{B_{t_i}}$

(at most $2\sqrt{g}$ elements)
 if $t_i < s$ then $M[i - d] \leftarrow M[i - d] + 1$
 end
 end

[The vector M is now correct since the first part of the algorithm included all the errors between blocks and the last part found all the errors within a block.]

end Algorithm

Time: $O(g \log g)$ for sorting
 $O(n\sqrt{g} \log m)$ for less-than matching of the
 representatives
 $O(n\sqrt{g})$ for correcting mismatches within
 blocks
Total: $O(n\sqrt{g} \log m)$

5.1. The Case of Lengths

In our case the numbers represent lengths of streams; therefore, we are guaranteed that following each number i in the pattern there are at least i don't cares.

In order to solve the less-than matching problem under these circumstances we first eliminate all the cases where a match of P in T causes one of the numbers $1, 2, \dots, m^{1/3}$ in the pattern to mismatch with the corresponding text position. This can be done by $m^{1/3}$ convolutions for a total time of $O(nm^{1/3} \log m)$.

We now have to eliminate the cases where there are mismatches in the remaining numbers. Exchange all numbers in the text and pattern that are not greater than $m^{1/3}$ by ϕ 's. The smallest remaining number is $m^{1/3} + 1$ and it is followed by $m^{1/3} + 1$ ϕ 's. Thus there are at most $m^{2/3}$ non- ϕ elements in the pattern ($g = m^{2/3}$).

By the previous section we can find all locations where the new pattern is less than the new text in time $O(n\sqrt{m^{2/3}} \log m) = O(nm^{1/3} \log m)$.

The intersection of all these locations and the ones where there is no mismatch of the numbers $1, \dots, m^{1/3}$ gives the required result.

6. THE APPROXIMATION PROBLEM

We have seen an $o(nm)$ algorithm for solving the swapped matching problem as a generalized matching problem. This algorithm can easily be adapted for solving the approximation problem with the number of swaps as the distance metric. Until now, the definition we gave for swapped matching was a generalized matching definition. However, we can view a swap

permutation operationally as resulting from a sequence of swaps of adjacent pairs, with no symbol participating in more than one swap. We are now interested in counting the number of swap operations that a text location needs to undergo in order to equal the pattern. Let us, for the moment, restrict ourselves to the swapped pattern matching with internal endpoints problem. It is clear that the number of such operations is exactly half the numbers of mismatches between the text and its swapped version (recall that we never swap a pair of equal symbols). For the general case, where the allowed swaps are not only internal, we need to count half of the mismatches resulting from internal swaps and then take care of the start and end points in case external swaps occurred there. Formally,

DEFINITION. Let $T = t_1, \dots, t_n$ be a text string over alphabet Σ . Let π be a swap permutation for T . The number of swaps in $\pi(T)[i, j]$ (the substring $t_{\pi(i)}, \dots, t_{\pi(j)}$) is $\lfloor \frac{1}{2} d(T[i, j], \pi(T)[i, j]) \rfloor + L + R - B$, where $d(T[i, j], \pi(T)[i, j])$ is the number of mismatches between $T[i, j]$ and $\pi(T)[i, j]$,

$$L = \begin{cases} 1, & \text{if } \pi(i) = i - 1; \\ 0, & \text{otherwise.} \end{cases}, \quad R = \begin{cases} 1, & \text{if } \pi(j) = j + 1; \\ 0, & \text{otherwise.} \end{cases},$$

and

$$B = \begin{cases} 1, & \text{if } R = L = 1; \\ 0, & \text{otherwise.} \end{cases}$$

The pattern matching with swaps metric problem is the following:

Input: Text string $T = t_1, \dots, t_n$ and pattern string $P = p_1, \dots, p_m$ over alphabet Σ .

Output: For every location i where P has a swapped match in T , where T' is the swapped version of T , write the number of swaps in $T'[i, i + m - 1]$.

THEOREM 3. The swaps metric problem can be solved in $O(n\sqrt{m} \log^2 m)$ steps.

Proof. Pattern matching with mismatches as a metric is well known to run in $O(n\sqrt{m} \log^2 m)$ time [1, 14]. Now, consider a text location where the pattern matches with k mismatches and the pattern also has a swapped matching. If the swapped match has internal endpoints then the number of swaps is exactly $\frac{k}{2}$. If exactly one of the endpoints swaps externally then the number of swaps is $\frac{k+1}{2}$. If both swap externally there are $\frac{k+2}{2}$ swaps. It is straightforward to adapt the solution to the swapped match problem to announce for each swapped match whether it matches with an external swap on either endpoint. Therefore we can implement pattern matching with swaps as a metric in $O(n\sqrt{m} \log^2 m)$ time. ■

ACKNOWLEDGMENTS

The authors kindly thank Alex Schäffer for sharing with us the biological inspiration and Ref. [19]. We are also indebted to a number of people, known and anonymous—the first being Pyotr Indyk—for pointing out to us Ref. [25].

REFERENCES

1. K. Abrahamson, Generalized string matching, *SIAM J. Comput.* **16** (1987), 1039–1051.
2. A. Amir and M. Farach, Efficient 2-dimensional approximate matching of half-rectangular figures, *Inform. and Comput.* **118** (1995), 1–11.
3. A. Amir, G. Landau, M. Lewenstein, and N. Lewenstein, Efficient special cases of swapped matching, *Inform. Process. Lett.* **68**, No. 3, (1997), 125–132.
4. A. Amir, M. Lewenstein, and E. Porat, Faster string matching with k mismatches, in “Proceedings of the Eleventh Symposium on Discrete Algorithms, San Francisco, CA,” 2000, 794–803.
5. R. S. Boyer and J. S. Moore, A fast string searching algorithm, *Comm. Assoc. Comput. Mach.* **20** (1977), 762–772.
6. V. Chvatal, D. A. Klarner, and D. E. Knuth, “Selected Combinatorial Research Problems,” Technical Report STAN-CS-72-292, Stanford University, 1972.
7. R. Cole and R. Hariharan, Approximate string matching: A faster simpler algorithm, in “Proc. 9th ACM-SIAM Symposium on Discrete Algorithm (SODA),” pp. 463–472, 1998.
8. M. J. Fischer and M. S. Paterson, String matching and other products, “Complexity of Computation” (R. M. Karp, Ed.), SIAM-AMS Proceedings, Vol. 7, pp. 113–125, 1974.
9. Z. Galil, Open problems in stringology, in “Combinatorial Algorithms on Words,” (Z. Galil and A. Apostolico, Eds.), Vol. 12, pp. 1–8, Springer-Verlag, Berlin/New York.
10. Z. Galil and R. Giancarlo, Improved string matching with k mismatches, *SIGACT News* **17** (1986), 52–54.
11. Z. Galil and K. Park, An improved algorithm for approximate string matching, *SIAM J. Comput.* **19** (1990), 989–999.
12. P. Inkyk, Deterministic superimposed coding with applications to pattern matching, in “Proc. 38th IEEE FOCS,” pp. 127–136, 1997.
13. H. Karloff, Fast algorithms for approximately counting mismatches, *Inform. Process. Lett.* **48**, No. 2, (1993), 53–60.
14. S. Rao Kosaraju, Efficient string matching, manuscript, 1987.
15. G. M. Landau and U. Vishkin, Efficient string matching with k mismatches, *Theoret. Comput. Sci.* **43** (1986), 239–249.
16. G. M. Landau and U. Vishkin, Fast parallel and serial approximate string matching, *J. Algorithms* **10** (1989), 157–169.
17. G. M. Landau, E. W. Myers, and J. P. Schmidt, Incremental string comparison, *SIAM J. Comput.* **27** (1998), 557–582.
18. V. I. Levenshtein, Binary codes capable of correcting, deletions, insertions and reversals, *Soviet Phys. Dokl.* **10** (1966), 707–710.
19. B. Lewin, Genes for SMA: Multum in parvo, *Cell* **80** (1995), 1–5.
20. R. Lowrance and R. A. Wagner, An extension of the string-to-string correction problem, *J. Assoc. Comput. Mc* **22** (1975), 177–183.

21. S. Muthukrishnan, New results and open problems related to non-standard stringology, in "Proc. 6th Combinatorial Pattern Matching Conference," Lecture Notes in Computer Science, Vol. 937, pp. 298–317, Springer-Verlag, Berlin/New York, 1995.
22. S. Muthukrishnan, Boolean convolution reduction to swapped matching, personal communication, 1997.
23. S. Muthukrishnan and K. Palem, Non-standard stringology: Algorithms and complexity, in "Proc. 26th Annual Symposium on the Theory of Computing," pp. 770–779, 1994.
24. S. Muthukrishnan and H. Ramesh, String matching under a general matching relation, *Inform. and Comput.* **122** (1995), 140–148.
25. J. (Seffi) Naor and M. Naor, Small-bias probability spaces: efficient constructions and applications, *SIAM J. Comput.* **22** (1993), 838–856.
26. A. Pentland, Invited talk, NSF Institutional Infrastructure Workshop, 1992.
27. S. C. Sahinalp and U. Vishkin, Efficient approximate and dynamic matching of patterns using a labeling paradigm, in "Proc. 37th FOCS," pp. 320–328, 1996.
28. E. Ukkonen, A linear-time algorithm for finding approximate shortest common superstrings, *Algorithmica* **5** (1990), 313–323.
29. R. A. Wagner, On the complexity of the extended string-to-string correction problem, in "Proc. 7th ACM STOC," pp. 218–223, 1975.